Dropfront TDD

This document covers how Dropfront is built. The rules themselves live in the GDD tab.

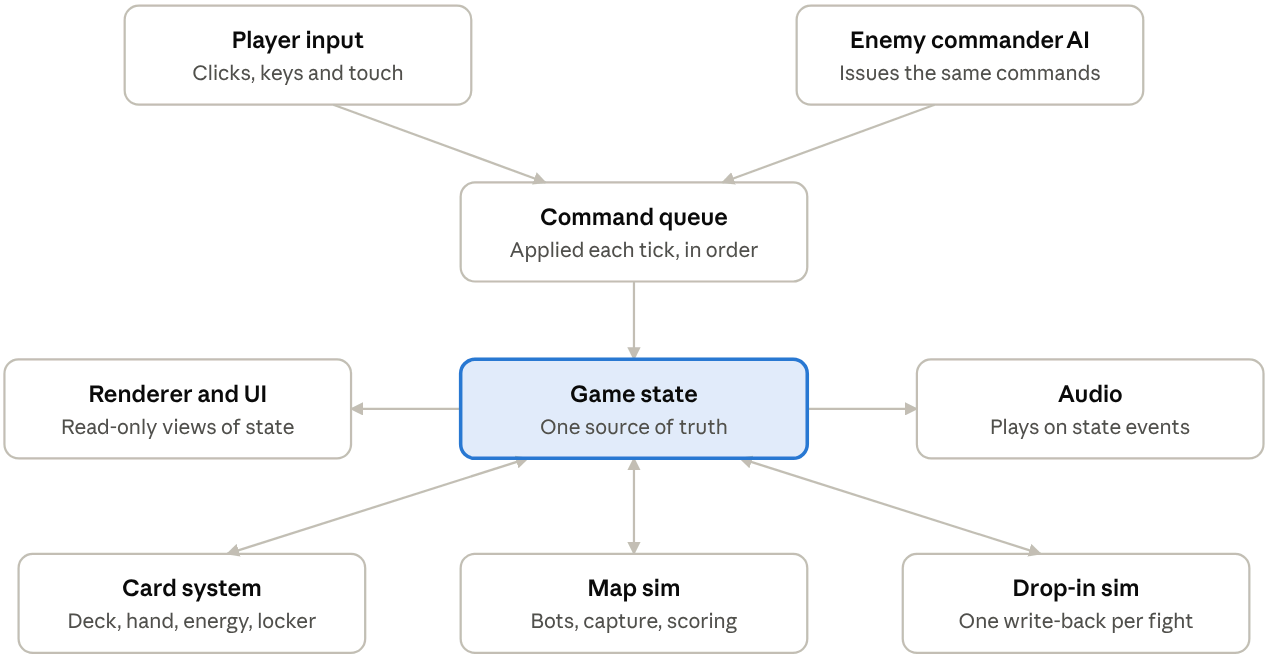
Tech choices

Area	Choice	Why
Platform	Browser, shipped as a static site (itch.io first)	No install, and the prototype already runs there
Language	TypeScript, bundled with Vite	Types catch card and state mistakes; instant reloads
Rendering	Canvas 2D, no engine	The art is flat shapes; the prototype proves it is fast enough
UI	Plain DOM over the canvas (HUD, hand, gear locker)	Text, focus and touch come for free
Simulation	Fixed 60 Hz tick, decoupled from rendering	The same inputs always give the same result
Randomness	One seeded RNG for the whole match	Replays, bug reports and balance tests can be rerun
Audio	Web Audio with a small sound bank	Starts on the first click, as browsers require
Saves	None in a match; run progress in localStorage	A lost run is a small loss

Architecture

One plain-data game state holds the whole match. Player input and the enemy AI both turn into commands, so the AI can never do something the player can't.

Every change goes through one queue into one game state



Systems read and write state only inside the tick. Views never write.

architecture · 8 modules, 1 state

- **Command queue:** `PlayCard`, `Scrap`, `EquipGear`, `DropIn`, `ChampionDropIn`. The tick applies commands first, then runs the systems.
- **Mode switch:** the state carries `mode: map | dropIn | over`. In `dropIn` the map sim is paused, and only the drop-in sim and the enemy champion run.
- **Testing:** since the sim never touches the DOM, a headless runner can play thousands of seeded matches overnight to check balance and match length.

Data and key algorithms

Cards are data

Each card is one record. Its effect is a small function, or a list of shared effect steps (spawn bots, freeze meter, draw), so adding a card is one entry and no new systems.

```
type Card = {
  id: string; name: string; cost: number;
  type: 'squad' | 'tactic' | 'gear';
  target: 'any' | 'held' | 'enemy' | 'fought' | 'nextToHeld' | null;
  exhaust?: boolean;
  play?: (s: GameState, point?: PointId) => void; // squads and tactics
  gear?: GearId;                                  // gear cards unlock this
};
```

Map sim, per tick

1. Move squads along their path; arrivals join the point's garrison.
2. **Combat:** if both sides are on a point, each side takes `0.11 x enemy strength x dt` damage, and every whole point of damage removes one bot.
3. **Capture:** the meter moves by `10 + 3 x bots` per second for a lone side, or by `2 x (yours - theirs)` when both are there. Skip frozen and locked points.
4. **Score:** each owned, unsabotaged point adds `2 x dt`. At 2000, or at 8:00 on the real-time clock, the match ends.

Enemy commander

- Every decision, score each point with weights for ownership, threat, distance and a seeded random term. Pick the best, then the cheapest command that serves it.
- Champion drop-in: a cooldown plus a trigger check (4 or more of your bots on its point, or its capture past halfway), and 2 of its energy.
- All weights sit in one tuning table, so balance changes never touch code.

Drop-in write-back

`resolveDropIn(state, pointId, result)` is the only function that turns a fight into map state. It sets the garrisons to the survivors plus anyone who sat out, flips and locks the point on a clear, and starts the champion's cooldown if it fell. This keeps the two simulations from disagreeing.

Implementation plan

The browser prototype proves the loop in one JavaScript file. The ticked items are already in it.

Week 1: both halves playable

- [x] ~~Map graph, bots walking, capture meters and scoring~~
- [x] ~~Cards, energy, 15-second turns and target highlights~~
- [x] ~~20-second drop-in that feeds survivors back to the map~~
- [x] ~~Enemy commander v1 that scores points~~
- [x] ~~Gear on the hero and scrap for energy~~
- [ ] Port to TypeScript and Vite with a fixed tick, seeded RNG and the command queue

Week 2: deck, gear and AI

- [ ] Cards as data, with a shared effects library
- [ ] Gear in play (2), discard to swap, and map at 25% during drop-ins
- [ ] Enemy commander v2: its own energy, mirrored tactic cards, tuning table
- [ ] Headless runner; tune scoring until most matches land at 5 to 8 minutes

Week 3: the firefight

- [ ] Enemy champion in the arena (15 HP, dash, 2 gear)
- [ ] Three enemy types: rifle, rusher, heavy
- [ ] Hit-stop, screen shake, capture and loss feedback
- [ ] Touch controls pass

Week 4: the run

- [ ] 3-match run, card draft after each win, AI ramp per match
- [ ] Title, results and run-summary screens
- [ ] Sound pass
- [ ] Playtest with 3 people, fix, ship to itch.io

Cut if behind: drop the run and the draft, and ship one match with a fixed deck.

Technical risks

Risk	What goes wrong	Mitigation
Two simulations drift	The map and a drop-in disagree on bot counts	All results go through `resolveDropIn`; headless tests compare totals before and after
Match length misses 5 to 8 minutes	Snowballs end games in 3 minutes, stalemates run 12	Headless runner reports the length spread; tune score rate and win target from the tuning table
Enemy AI feels random	Its moves don't read as plans	Log each decision and its top 3 scores; show the AI's target briefly on the map while testing
Touch controls	Twin-stick shooting is hard on a phone	Auto-aim and hold-to-move first; cut touch to the map only if it doesn't feel good by week 3
Performance on low-end laptops	Too many bullets or bots drop frames	Cap units per arena, pool bullets, draw each frame from state with no allocations